Quantifying the Blast Radius: Centrality-Based Criticality Modeling for Infrastructure Risk

Most companies think they know which systems are critical. They have spreadsheets, risk registers, architecture diagrams with color-coded tiers. What those documents almost never capture is the difference between what a system does and what depends on it. A payment database might serve a single application. That application might serve twelve others. Those twelve might collectively handle every customer transaction the company processes. The database is critical. The register says otherwise.

This project exists to close that gap.

NexaCore Financial Technologies runs 90 distinct software systems: payment processors, authentication services, databases, message queues, third-party vendors, monitoring tools, and more. Each one depends on others to function. This project models those dependencies as a directed graph – a network where each system is a node and each dependency is an arrow pointing from the dependent to the thing it depends on. Once the network is built, three mathematical techniques answer the same question from three different angles: which node, if it disappeared, would cause the most damage?

The three techniques are called centrality measures. Betweenness centrality identifies nodes that sit on the most paths between other nodes – the routing chokepoints that everything passes through. Eigenvector centrality identifies nodes that are connected to other well-connected nodes – the systems at the structural center of the network. PageRank, originally designed to rank web pages, identifies nodes that accumulate dependency weight from the structure of what points to them – the foundational systems that everyone ultimately relies on.

Using three measures rather than one is deliberate. A single score would hide information. When all three measures agree that a node is critical, the finding is structurally airtight. When they disagree, the disagreement is the finding: a node can be critical for different structural reasons, and those reasons point to different fixes. A routing chokepoint needs a redundant path. A foundational database needs a read replica. Averaging the scores together would obscure that distinction.

The analysis identified several nodes that no standard risk assessment would have flagged as single points of failure. The Authentication Gateway – the system through which every user login passes – has no backup path. Its removal breaks 285 of the 566 reachable connections in the entire network, more than any other node by a wide margin. The Secrets Vault, which stores database credentials for every service, ranks first on two of the three centrality measures. The Shared Message Queue, through which 14 services communicate, scores highly on all three. None of these systems appear in NexaCore's formal SPOF register.

The most instructive finding involves a third-party vendor that handles customer identity verification. Every centrality measure ranked it near the bottom – 74th out of 90. A direct simulation, removing the node and counting what broke, told a different story. Three workflows stop completely when the vendor is unavailable: client onboarding, regulatory compliance reporting, and real-time fraud screening. There is no cached fallback, no alternative provider, no circuit breaker. Centrality analysis would have sent this vendor to the bottom of the remediation list. Simulation moved it to the top ten.

That gap between mathematical ranking and operational consequence is the project's core argument. Centrality measures are fast, systematic, and independent of whoever last updated the risk register. They identify structural importance from the graph alone. What they can't see is a leaf node with three blocking dependencies and no fallback. Blast radius simulation sees exactly that. Used together, the two methods produce a remediation backlog that is both mathematically defensible and operationally grounded – which is what distinguishes a risk analysis from a risk register.